

L7 Limbajul SQL

2. Funcții referitoare la mai multe înregistrări (multiple-row functions)

1. Obiective

- A. Multiple-row functions
- B. Multiple-row functions - folosind interfața grafică a aplicației Oracle SQL Developer
- C. Cereri din mai multe tabele (JOIN-uri)
 - Regăsirea datelor din două sau mai multe tabele
 - Joncțiuni

2. Considerații teoretice

A. Funcții referitoare la mai multe înregistrări (multiple-row functions)

Aceste funcții se mai numesc și funcții **totalizatoare** sau funcții **de grup**. Spre deosebire de funcțiile referitoare la o singură înregistrare, funcțiile de grup operează pe un set de mai multe înregistrări și returnează un singur rezultat pentru fiecare grup. Dacă nu este utilizată clauza GROUP BY, ce grupează înregistrările după un anumit criteriu, tabela este considerată ca un singur grup și se va returna un singur rezultat.

Funcție	Efect
COUNT	determină numărul de înregistrări care îndeplinesc o anumită condiție
MAX	determină cea mai mare valoare dintr-o coloană
MIN	determină cea mai mică valoare dintr-o coloană
SUM	returnează suma tuturor valorilor dintr-o coloană
AVG	calculează valoarea medie a unei coloane
STDDEV	determină abaterea sau deviația standard a unei coloane numerice
VARIANCE	returnează dispersia, adică pătratul unei deviații standard pentru o coloană numerică

Exemplu:

```
SQL>SELECT MIN(salariu.),MAX(salariu),AVG(salariu),COUNT(*)  
FROM profesor;
```

MIN(SALARIU)	MAX(SALARIU)	AVG(SALARIU)	COUNT(*)
-----	-----	-----	-----
1200	3000	2057.1429	7

Toate funcțiile de mai sus, cu excepția funcției COUNT, operează asupra unei coloane sau unei expresii, care este specificată ca parametri ai funcției. În cazul funcției COUNT, argumentul acesteia nu contează, de obicei utilizându-se ca argument simbolul *.

Exemple:

1. **SELECT** MIN(salariu),MAX(salmax),MIN(salminim),COUNT(*)
FROM angajati;

Rezultat:

```
SQL> select min(salariu),max(salmax),min(salminim),count(*)
  2  from angajati;

MIN(SALARIU) MAX(SALMAX) MIN(SALMINIM) COUNT(*)
-----
          2000          4500          1600          5
```

2. **SELECT** MAX(salmax)
FROM angajati;

Rezultat:

```
SQL> select max(salmax)
  2  from angajati;

MAX(SALMAX)
-----
          4500
```

3. **SELECT** MIN(salariu)
FROM angajati;

Rezultat:

```
SQL> select min(salariu)
  2  from angajati;

MIN(SALARIU)
-----
          2000
```

4. **SELECT** SUM(salminim)
FROM angajati;

Rezultat:

```
SQL> select sum(salminim)
  2  from angajati;

SUM(SALMINIM)
-----
          16300
```

5. **SELECT** AVG(salariu)
FROM angajati;

Rezultat:

```
SQL> select avg(salariu)
      2 from angajati;

AVG(SALARIU)
-----
      3404
```

6. **SELECT** STDDEV(salmax)
FROM angajati;

Rezultat:

```
SQL> select stddev(salmax)
      2 from angajati;

STDDEV(SALMAX)
-----
    992.108865
```

7. **SELECT** VARIANCE(salminim)
FROM angajati;

Rezultat:

```
SQL> select variance(salminim)
      2 from angajati;

VARIANCE(SALMINIM)
-----
    1063000
```

Observație: Toate funcțiile de mai sus ignoră valorile Null, excepție făcând funcția COUNT. Pentru a include în calcule și înregistrările cu valoarea Null se poate folosi funcția NVL.

Dacă nu este utilizată clauza GROUP BY, în lista de attribute ale comenzii SELECT nu pot apare funcții de grup alături de nume de coloane sau alte expresii care iau valori pentru fiecare înregistrare în parte.

De exemplu, următoarea întrebare va genera o eroare:

```
SQL> SELECT nume, KIN (salariu)
FROM profesor;
```

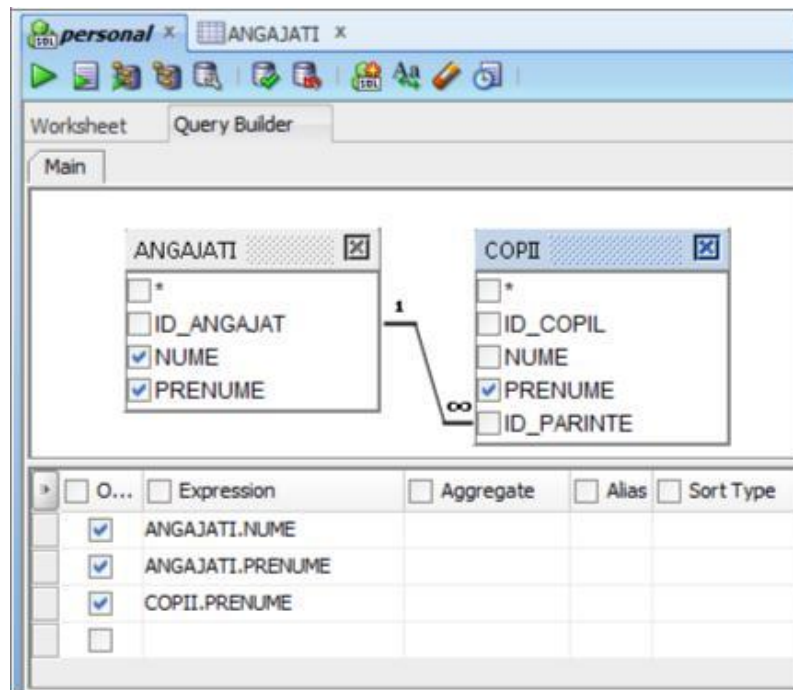
ERROR at line 1:

ORA-00937: not a single-group group function

B. Selectarea din mai multe tabele (Funcții referitoare la mai multe înregistrări (multiple-row functions) - folosind interfața grafică a aplicației Oracle SQL Developer

Principial o bază de date relațională presupune repartizarea datelor în mai multe tabele. Așa cum s-a văzut deja, acest mod de stocare permite eliminarea informațiilor redondante. Pentru a găsi articolele dintr-un tabel în relație cu înregistrări din alt tabel trebuie însă inclusă în frazele SELECT condiția de legătură dintre cele două tabele. Reprezentarea grafică reprezentând tabelele bazei de date și relațiile dintre ele poartă numele de **diagramă E-R** (entități-relații, eng. *entity-relationship*).

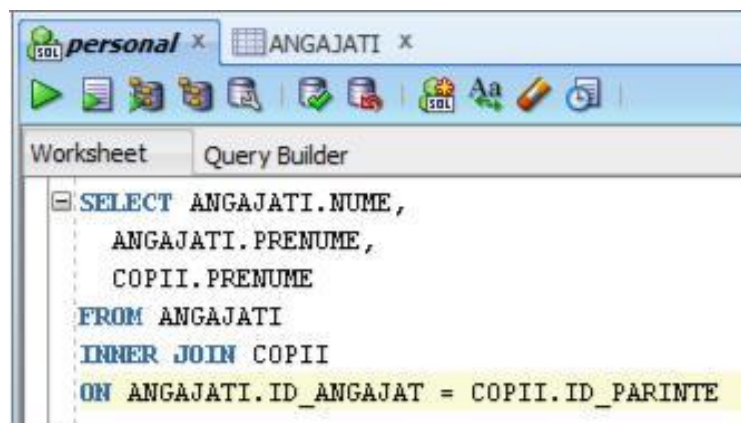
Exemplu fundamental:



The screenshot shows the **Query Result** window, displaying the results of the SQL query. The window title is "Query Result x". The status bar indicates "All Rows Fetched: 3 in 0 seconds". The results are shown in a table with four columns: **NUME**, **PRENUME**, and **PRENUME_1**. The data is as follows:

	NUME	PRENUME	PRENUME_1
1	Pop	Vasile	Marius
2	Ionescu	Valer	Mircea
3	Ionescu	Valer	Alexandru

Comanda SQL corespunzătoare este următoarea:



Comanda generată conține clauza suplimentară *JOIN*. Aceasta permite definirea legăturilor existente între tabele.

1. Legăturile dintre tabele pot fi impuse și folosind clauza **WHERE**, ca în exemplul următor:

```
SELECT Edituri.Nume, Carti.Titlu, Carti.Anul
FROM Edituri, Carti
WHERE Nume='Minerva' AND Edituri.Code=Carti.Code
```

A doua condiție conținută în clauza **WHERE** este cea care leagă tabelele *Edituri* și *Carti*.

2. Folosind **JOIN**, exemplul anterior poate fi scris astfel:

```
SELECT Edituri.Nume, Carti.Titlu, Carti.Anul
FROM Edituri INNER JOIN Carti ON Edituri.Code=Carti.Code
WHERE Nume='Minerva'
```

Clauza *JOIN* este însă singura soluție într-o altă situație, respectiv când într-un tabel se acceptă pentru o cheie străină valori nule.

Un *exemplu* simplu este cel al tabelelor *Angajati* și *Soți* prezentate în figura 7.1.

ANGAJATI					SOTI		
EDIT	ID_ANG	NUME	PRENUME	ID_SOT	EDIT	NUME_PRENUME	ID_SOT
	1	Avram	Lucian	-		Ionescu Marcel	2
	2	Breban	Ioan	-		Popescu Simona	1
	5	Ionescu	Carmen	2	row(s) 1 - 2 of 2		
	6	Banu	Mircea	-			
	4	Anghel	Mihai	-			
	3	Popescu	Ilie	1			
row(s) 1 - 6 of 6							

Figura 7.1. Folosire JOIN

Dintre angajații din primul tabel doi sunt căsătoriți, în cazul lor ID_SOT are valori nenule.

Dacă se dorește realizarea unei mulțimi de selecție care să cuprindă angajații, iar pentru cei căsătoriți numele și prenumele soțului/soției, fraza *SELECT* ar putea fi următoarea:

```
SELECT Angajati.Nume, Angajati.Prenume, Soti.Nume_prenume  
FROM Angajati, Soti  
WHERE Angajati.id_sot=Soti.id_sot
```

Rezultat:

NUME	PRENUME	NUME_PRENUME
Ionescu	Carmen	Ionescu Marcel
Popescu	Ilie	Popescu Simona

Rezultatul nu este probabil cel dorit deoarece prin condiția pusă sunt excluși din mulțimea de selecție angajații necăsătoriți.

Soluția este oferită de clauza **LEFT OUTER JOIN**:

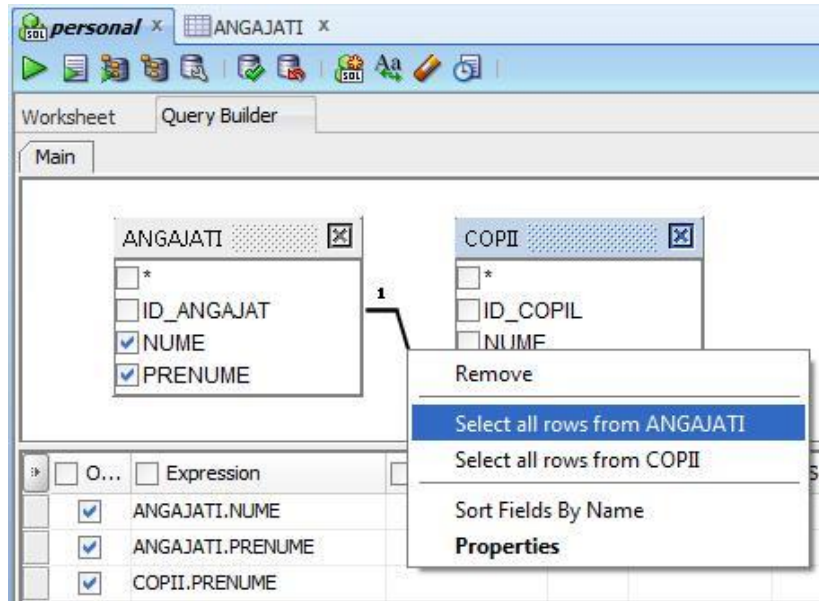
```
SELECT Angajati.Nume, Angajati.Prenume, Soti.Nume_prenume  
FROM Angajati LEFT OUTER JOIN Soti on Angajati.id_sot=soti.id_sot
```

Rezultatul va cuprinde toate liniile din primul tabel iar pentru liniile din primul tabel care au correspondent în al doilea tabel, va include și informațiile corespunzătoare din al doilea tabel.

Similar se poate folosi clauza **RIGHT OUTER JOIN** pentru includerea tuturor înregistrărilor din al doilea tabel și selectarea din primul doar a celor care satisfac relația de legătură.

NUME	PRENUME	NUME_PRENUME
Avram	Lucian	-
Breban	Ioan	-
Ionescu	Carmen	Ionescu Marcel
Banu	Mircea	-
Anghel	Mihai	-
Popescu	Ilie	Popescu Simona

Observație: Pentru a impune în *Query Builder* tipul de clauză *JOIN* dorit se selectează cu butonul drept al mouse-ului linia care unește cele două tabele și se indică folosind meniul contextual, soluția dorită.



Limbaajul SQL permite înlocuirea unei mulțimi care contribuie la selecție printr-o altă frază *SELECT*, ca în exemplul următor.

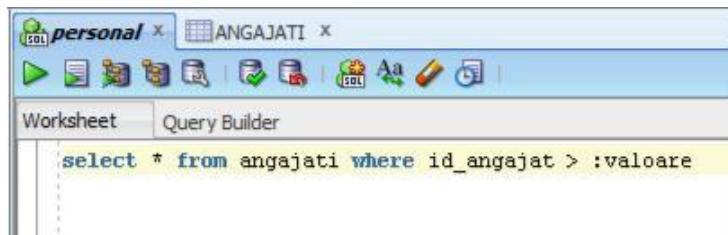
```
SELECT Edituri.Nume, Carti.Titlu, Carti.Anul FROM Edituri, Carti
WHERE Nume='Minerva' AND Edituri.Code=Carti.Code AND
Titlu.Data IN (SELECT Data
FROM Carti
WHERE Data BETWEEN '12-MAY-1975' AND '29-MAY-1980')
```

Utilizarea parametrilor

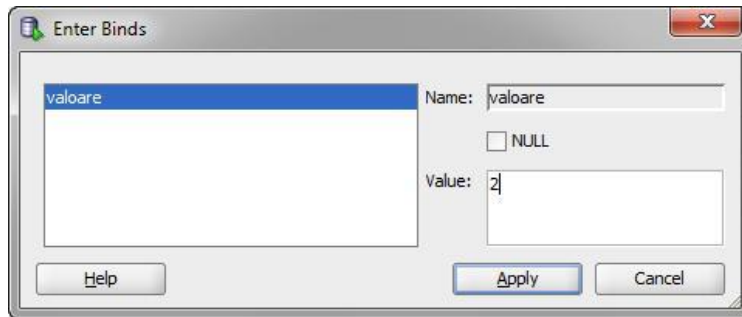
SQL permite ca la scrierea comenzii *SELECT* să se folosească parametrii. Valorile acestora sunt cerute într-o fereastră separată, în momentul executării interogării.

Exemplu:

```
SELECT * FROM angajati WHERE id_angajat > :valoare
```



Aplicația va afișa o fereastră de dialog în care va cere definirea valorii parametrului “valoare”:



Rezultat:

Query Result x		
SQL All Rows Fetched: 1 in 0.093 seconds		
NUME	PRENUME	ID_ANGAJAT
1 Pop	Vasile	3

C. CERERI din mai multe tabele (JOIN-uri)

Regăsirea datelor din două sau mai multe tabele

O joncțiune este o interogare care regăsește înregistrări din două sau mai multe tabele. Capacitatea de a realiza o joncțiune între două sau mai multe tabele reprezintă una dintre cele mai puternice facilități ale unui sistem relațional. Legătura dintre înregistrările tabelor se realizează prin existența unor câmpuri comune caracterizate prin domenii de definiție compatibile (chei primare sau străine). Pentru realizarea unei joncțiuni se folosește comanda SELECT, precizând în clauza FROM numele tabelor utilizate, iar în clauza WHERE criteriul de compunere.

Produsul a două sau mai multe tabele

În cazul în care în interogare se specifică mai multe tabele și nu este inclusă o clauză WHERE, interogarea va genera produsul cartezian al tabelor. Acesta va conține toate combinațiile posibile de înregistrări din tabelele componente. Astfel, produsul cartezian a două tabele care conțin 100, respectiv 50 de înregistrări va avea dimensiunea de 5.000 de înregistrări.

De exemplu, să considerăm tabela **DEPARTAMENT** cu următoarele 4 înregistrări:

COD_DEPARTAMENT	NUME	PROFIL
10	INFORMATICA	TEHNIC
20	ELECTRONICA	TEHNIC
30	AUTOMATICA	TEHNIC
40	FINANTE	ECONOMIC

Atunci următoarea interogare va genera produsul cartezian al tabelelor, adică va avea ca rezultat $7 \times 4 = 28$ de rânduri ce vor conține toate combinațiile posibile de înregistrări din cele două tabele:

```
SOL> SELECT *
      FROM profesor, DEPARTAMENT;
```

Dacă în lista de attribute ale comenzii SELECT sunt specificate coloanele selectate, atunci numele acestora trebuie să fie unice în cadrul tuturor tabelelor. Dacă există un nume de coloană care apare în mai mult de un tabel, atunci, pentru evitarea ambiguității, trebuie specificat și tabelul din care face parte coloana în cauză.

De exemplu, în următoarea interogare pentru coloanele "cod_DEPARTAMENT" și "nume" trebuie specificate tabelele din care fac parte:

```
SQL> SELECT profesor.nume, prenume,
      departament.cod_departament, departament.nume
      FROM profesor, departament;
```

Rezultat:

NUME	PRENUME	COD DEPARTAMENT	NUME
----	-----	-----	----
GHEORGHIU	STEFAN	10	INFORMATICA
MARIN	VLAD	10	INFORMATICA
GEORGESCU	CRISTIANA	10	INFORMATICA
IONESCU	VERONICA	10	INFORMATICA
ALBU	GHEORGHE	10	INFORMATICA
VOINEA	MIRCEA	10	INFORMATICA
STANESCU	MARIA	10	INFORMATICA
GHEORGHIU	STEFAN	20	ELECTRONICA
MARIN	VLAD	20	ELECTRONICA
GEORGESCU	CRISTIANA	20	ELECTRONICA
IONESCU	VERONICA	20	ELECTRONICA
ALBU	GHEORGHE	20	ELECTRONICA
VOINEA	MIRCEA	20	ELECTRONICA
STANESCU	MARIA	20	ELECTRONICA
GHEORGHIU	STEFAN	30	AUTOMATICA
MARIN	VLAD	30	AUTOMATICA
GEORGESCU	CRISTIANA	30	AUTOMATICA
IONESCU	VERONICA	30	AUTOMATICA
ALBU	GHEORGHE	30	AUTOMATICA
VOINEA	MIRCEA	30	AUTOMATICA
STANESCU	MARIA	30	AUTOMATICA
GHEORGHIU	STEFAN	40	FINANTE
MARIN	VLAD	40	FINANTE
GEORGESCU	CRISTIANA	40	FINANTE
IONESCU	VERONICA	40	FINANTE

ALBU	GHEORGHE	40	FINANTE
VOINEA	MIRCEA	40	FINANTE
STANESCU	MARIA	40	FINANTE

În general, pentru a scurta textul comenzii, în astfel de cazuri se folosesc de obicei alias-uri pentru numele tabelor, care pot fi folosite în interogare. Astfel interogarea de mai sus se mai poate scrie:

```
SQL> SELECT p.num, prenume, c.cod_departament, c.num
FROM profesor p, departament c;
```

În general, produsul cartezian este rar folosit, având o utilitate practică redusă.

Joncțiuni

Pentru a realiza o joncțiune între două sau mai multe tabele se utilizează clauza WHERE a interogărilor pe aceste tabele, în funcție de criteriul de compunere, se disting mai multe tipuri de joncțiuni:

1. joncțiuni echivalente (EQUI-JOIN) sau joncțiuni interne (INNER JOIN);
2. joncțiuni neechivalente;
3. joncțiuni externe (OUTER JOIN);
4. autojoncțiuni.

1. Joncțiunile echivalente

O echijoncțiune conține operatorul egalitate (=) în clauza WHERE, combinând înregistrările din tabele care au valori egale pentru coloanele specificate.

De *exemplu*, pentru a afișa cadrele didactice și numele departamentului din care aceștea fac parte se combină înregistrările din cele două tabele pentru care codul departamentului este același.

```
SQL> SELECT p.num, p.prenume, c.num 'NUME DEPARTAMENT'
FROM profesor p, departament c
WHERE p.cod DEPARTAMENT=c.cod DEPARTAMENT;
```

Rezultat:

NUME	PRENUME	NUME DEPARTAMENT
----	-----	-----
GHEORGHIU	STEFAN	INFORMATICA
IONESCU	VERONICA	INFORMATICA
VOINEA	MIRCEA	INFORMATICA
MARIN	STANESCU	ELECTRONICA
ALBU	GEORGESCU	ELECTRONICA
VLAD	MARIA	ELECTRONICA
GHEORGHE	CRISTIANA	AUTOMATICA

2. Joncțiuni neechivalente

Joncțiunile neechivalente sunt acelea care nu folosesc în clauza WHERE operatorul egal. Operatorii cei mai utilizați în cazul joncțiunilor neechivalente sunt: <, >, <=, >=, <>, BETWEENAND.

Pentru a *exemplifica* un astfel de tip de joncțiune considerăm tabela **gradsal** ce conține pragul minim și pragul maxim al salariului dintr-un anumit grad de salarizare:

GRAD	SALARIZARE	PRAG MIN	PRAG MAX
-----		-----	-----
1		500	1500
2		1501	2000
3		2001	2500
4		2501	3500
5		3501	10000

Evident, între tabelele **profesor** și **gradsal** nu are sens definirea unei joncțiuni echivalente deoarece nu există o coloană din tabela **profesor** careia să-i corespundă o coloană din tabela **gradsal**.

Exemplul următor ilustrează definirea unei joncțiuni neechivalente care evaluează gradul de salarizare a cadrelor didactice, prin încadrarea salariului acestora într-un interval stabilit de pragul minim și pragul maxim:

```
SQL> SELECT p.num, p.grad, p.salariu, g.grad_salarizare
      FROM profesor p, gradsal g
      WHERE p.salariu BETWEEN g.prag_min AND g.prag_max;
```

Rezultat:

NUME	GRAD	SALARIU	GRAD SALARIZARE
----	----	-----	-----
IONESCU	ASIST	1500	1
VOINEA	ASIST	1200	1
STANESCU	ASIST	1200	1
MARIN	PROF	2500	3
ALBU	LECT	2200	3
GHEORGHIU	PROF	3000	4
GEORGESCU	CONF	2800	4

3. Joncțiuni externe

Dacă într-o joncțiune de tipul celor prezentate până acum una sau mai multe înregistrări nu satisfac condiția de compunere specificată în clauza WHERE, atunci ele nu vor apărea în rezultatul interogării. Aceste înregistrări pot apărea însă dacă se folosește joncțiunea externă. Joncțiunea externă returnează toate înregistrările care satisfac condiția de joncțiune plus acele

înregistrări dintr-un tabel ale căror valori din coloanele după care se face legătura nu se regăsesc în coloanele corespunzătoare ale nici unei înregistrări din celălalt tabel.

Pentru a realiza o joncțiune externă între tabelele A și B ce returnează toate înregistrările din tabela A se utilizează semnul (+) în dreapta tabelului B. Pentru fiecare înregistrare din tabela A care nu satisface condiția de compunere pentru nici o înregistrare din tabela B, se va crea în tabela B o înregistrare nulă care va fi compusă cu înregistrarea din tabela A. Invers, pentru a realiza o joncțiune externă între tabelele A și B ce returnează toate înregistrările din tabela B, se utilizează semnul (+) în dreapta tabelului A.

În interogarea utilizată pentru a exemplifica joncțiunea echivalentă, se observă că au fost selectate numai departamentele în care există cadre didactice. Pentru a afișa toate departamentele, indiferent dacă ele cuprind sau nu cadre didactice, se folosește următoarea interogare:

```
SQL> SELECT p.num, p.prenume, c.num  
      FROM profesor p, DEPARTAMENT c  
      WHERE p.cod DEPARTAMENT(+) = c.cod DEPARTAMENT;
```

Rezultat:

NUME	PRENUME	NUME
----	-----	----
GHEORGIU	STEFAN	INFORMATICA
IONESCU	VERONICA	INFORMATICA
VOINEA	MIRCEA	INFORMATICA
MARIN	VLAD	ELECTRONICA
STANESCU	MARIA	ELECTRONICA
ALBU	GHEORGHE	ELECTRONICA
GEORGESCU	CRISTIANA	AUTOMATICA
FINANTE		

Se observă că ultima înregistrare (ce corespunde departamentului de finanțe care nu are în componență nici un cadru didactic) va avea coloanele corespunzătoare primului tabel completate cu Null.

Folosirea operatorului de joncțiune externă are următoarele restricții:

1. Operatorul (+) poate fi plasat în oricare parte a condiției din clauza WHERE, însă nu în ambele părți. Tabelul de partea căruia este amplasat acest operator va crea înregistrări nule care vor fi compuse cu înregistrările din celălalt tabel care nu satisfac condiția de compunere.
2. Dacă tabelele A și B au condiții multiple de joncțiune, atunci operatorul (+) trebuie utilizat în toate aceste condiții.
3. Într-o singură interogare nu se poate realiza o joncțiune externă a unui tabel cu mai multe tabele.
4. O condiție care conține operatorul (+) nu poate fi combinată cu o altă condiție ce utilizează operatorul IN.
5. O condiție care conține operatorul (+) nu poate fi combinată cu o altă condiție prin operatorul OR.

4. Auto-joncțiuni

Auto-joncțiunea reprezintă joncțiunea unui tabel cu el însuși. Pentru ca rândurile dintr-un tabel să poată fi compuse cu rânduri din același tabel, în clauza FROM a interogării numele tabelului va apare de mai multe ori, urmat de fiecare dată de un alias.

De *exemplu*, pentru a selecta toate cadrele didactice care au un șef direct și numele acestui șef se folosește următoarea auto-joncțiune:

```
SQL> SELECT p.num, p.prenume, s.num, s.prenume
      FROM profesor p, profesor s
      WHERE p.sef=s.cod;
```

Rezultat:

NUME	PRENUME	NUME	PRENUME
----	-----	-----	-----
MARIN	VLAD	GHEORGHIU	STEFAN
GEORGESCU	CRISTIANA	GHEORGHIU	STEFAN
ALBU	GHEORGHE	GHEORGHIU	STEFAN
VOINEA	MIRCEA	GHEORGHIU	STEFAN
IONESCU	VERONICA	GEORGESCU	CRISTIANA
STANESCU	MARIA	IONESCU	VERONICA

Auto-joncțiunea poate fi folosită și pentru verificarea corectitudinii interne a datelor. De *exemplu*, este puțin probabil să existe două cadre didactice care au cod diferit dar în schimb au același nume, prenume și data de naștere. Pentru a verifica dacă există astfel de înregistrări se folosește interogarea:

```
SQL> SELECT a.num, a.prenume
      FROM profesor a, profesor b
      WHERE a.num=b.num AND a.prenume=b.prenume AND a.data-
nast= b.data-nast AND a.cod<>b.cod
```

3. Desfășurarea lucrării

Exerciții rezolvate:

1. Să se afișeze numele salariatului și numele departamentului pentru toți salariații care au litera A inclusă în nume.

Soluție:

```
SELECT nume, denumire
FROM   angajati a, departamente d
WHERE  a.id_dept = d.id_dept AND lower(nume) LIKE '%a%';
```

2. Să se afișeze numele, funcția, codul și numele departamentului pentru toți angajații care lucrează în Timisoara.

Soluție:

```
SELECT nume, functie, a.id_dept, d.denumire
FROM   angajati a, departamente d, locatii l
WHERE  a.id_dept = d.id_dept
AND    d.id_locatie = l.id_locatie
AND    LOWER(oras)='timisoara';
```

3. Să se listeze numele, salariul și comisionul tuturor angajaților al căror salariu total (cu tot cu comision) depășește 10000\$.

Soluție:

```
SELECT nume, prenume, salariu, comision, salariu + salariu * NVL(comision,
0) FROM angajati
WHERE salariu + salariu * NVL(comision, 0) > 10000;
```

4. Creați o cerere care să afișeze numele angajatului, codul departamentului și toți salariații care lucrează în același departament cu el. Se vor eticheta coloanele corespunzător.

Soluție:

```
SELECT a.nume, a.id_dept, c.n
```

5. Să se afișeze numele și data angajării pentru salariații care au fost angajați după Popescu.

Soluție:

```
SELECT a.nume, a.data_ang
FROM   angajati a, angajati g
WHERE  LOWER(g.nume)='popescu' AND a.data_ang > g.data_ang;
```

6. Să se afișeze numele salariatului și data angajării împreună cu numele și data angajării șefului direct pentru salariații care au fost angajați înaintea șefilor lor. Se vor eticheta coloanele Angajat, Data_ang, Manager si Data_mgr.

7.

Soluție:

```
SELECT a.nume Angajat, a.data_ang Data_ang,
       m.nume Manager, m.data_ang Data_mgr
FROM   angajati a, angajati m
WHERE  a.id_manager = m.id_angajat AND a.data_ang < m.data_ang;
```

Probleme propuse spre rezolvare:

1. Scrieți o cerere care să întoarcă numele, numărul și numele departamentului pentru toți angajații.
2. Scrieți o cerere care să aibă ca rezultat toate slujbele care se găsesc în departamentul 20. Includeți și locația departamentului în rezultat.
3. Realizați o cerere care să întoarcă numele angajatului, numele departamentului, locația și slujba tuturor celor care câștigă un comision.
4. Scrieți o cerere care să întoarcă numele, numărul și numele departamentului și funcția tuturor celor care lucrează în Craiova.
5. Sa se selecteze toți angajații care fac parte din departamentul 'Productie', care s-au angajat înaintea angajatului din departamentul lor, care are cel mai mare salariu din departament.
6. Sa se afișeze numele fiecărui angajat, însoțit de numele departamentului din care face parte, data angajării lor.

Să se selecteze fiecare pentru fiecare departament, denumirea sa, numărul de angajați din departament, salariul mediu din departament, pentru toate departamentele cu mai mult de 3 angajați.

7. Introduceți câteva comenzi SQL, folosind și opțiunea de salvare a acestor comenzi:

- toți autorii în ordine alfabetică, afișând toate câmpurile tabelului;
`select * from autori order by nume asc`
- numele și prenumele autorilor în ordine alfabetică;
`select nume, prenume from autori order by nume asc`
- titlurile cărților din bibliotecă;
`select titlu from carti`
- titlurile cărților apărute în anul 1991;
`select titlu from carti where an_apar=1991`
- titlurile cărților apărute în anul specificat de utilizator (variabilă);
`select titlu from carti where an_apar=:an`
- titlul cărților și anul de apariție pentru cărțile care au codul > 2;
`select titlu, an_apar from carti where cod_carte>2`
- titlul cărții, anul apariției și codul cărții pentru cărțile apărute înainte de 1991 și care au codul < 2;
`select titlu, an_apar, cod_carte from carti where an_apar<=1991 and cod_carte>2`
- numele și prenumele cititorilor care au nume care încep cu Po;
`select nume, prenume from cititori where nume like 'Po%'`

- afișarea localităților de unde provin cititorii;

```
select distinct localitatea from cititori
```

- afișarea numărului de cărți din bibliotecă;

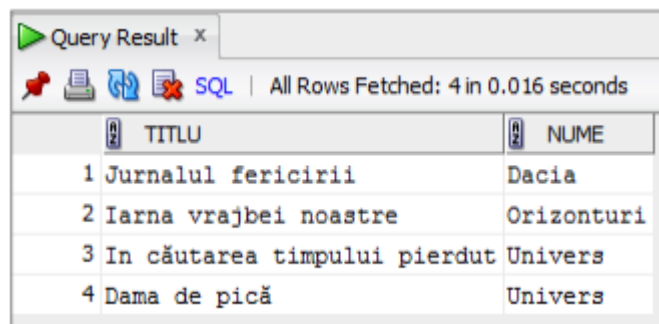
```
select count(*) as nr from carti
```

8. Introduceți și alte comenzi SQL pentru a afla date din celelalte tabele

9. Introduceți câteva comenzi SQL, folosind și opțiunea de salvare a acestor comenzi:

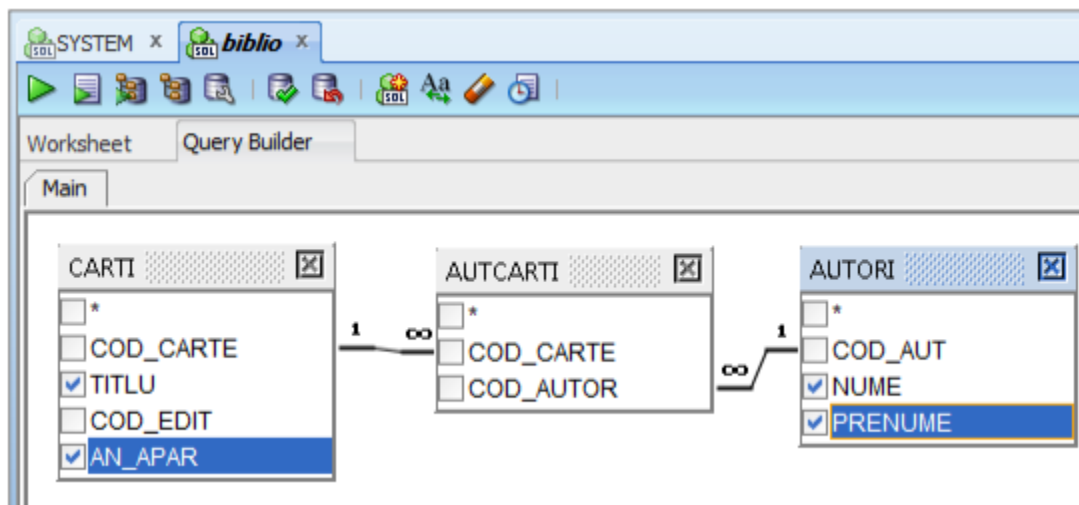
- selectarea cărților și a editurilor corespunzătoare

```
SELECT carti.titlu, edituri.ume FROM carti
INNER JOIN edituri ON edituri.cod_edit = carti.cod_edit
ORDER BY edituri.ume
```



	TITLU	NUME
1	Jurnalul fericirii	Dacia
2	Iarna vrajbei noastre	Orizonturi
3	In căutarea timpului pierdut	Univers
4	Dama de pică	Univers

- selectarea tuturor cărților și vizualizarea titlului, a anului apariției și a autorului:



```
SELECT carti.titlu, carti.an_apar, autori.ume,
autori.prenume FROM carti inner join autcarti
ON carti.cod_carte = autcarti.cod_carte
INNER JOIN autori ON autori.cod_aut = autcarti.cod_autor
```

- titlul cărții (în ordine alfabetică), numele și prenumele autorului și editura:

Query Result x			
SQL All Rows Fetched: 4 in 0 seconds			
TITLU	NUME	PRENUME	EDITURA
1 Dama de pică	Steinhardt	Nicolae	Univers
2 Iarna vrajbei noastre	Steinbeck	John	Orizonturi
3 In căutarea timpului pierdut	Proust	Marcel	Univers
4 Jurnalul fericirii	Puskin	Alexandr	Dacia

```
SELECT carti.titlu, autori.numa, autori.prenume, edituri.numa
AS editura FROM carti INNER JOIN autcarti ON carti.cod_carte =
autcarti.cod_carte
INNER JOIN AUTORI ON AUTORI.COD_AUT = AUTCARTI.COD_AUTOR
INNER JOIN EDITURI ON EDITURI.COD_EDIT = CARTI.COD_EDIT
ORDER BY CARTI.TITLU
```

Observație: În SQL || (două caractere “[”]) reprezintă operatorul de concatenare. Folosind acest operator se pot genera în mulțimea de selecție valori obținute prin concatenarea valorilor conținute în coloane distincte. Folosind acest operator, problema precedentă poate fi rezolvată astfel:

Query Result x			
SQL All Rows Fetched: 4 in 0 seconds			
TITLU	NUME_AUTOR	EDITURA	
1 Dama de pică	Steinhardt Nicolae	Univers	
2 Iarna vrajbei noastre	Steinbeck John	Orizonturi	
3 In căutarea timpului pierdut	Proust Marcel	Univers	
4 Jurnalul fericirii	Puskin Alexandr	Dacia	

```
SELECT carti.titlu, autori.numa || ' ' || autori.prenume as
nume_autor, edituri.numa as editura
FROM carti
INNER JOIN autcarti on carti.cod_carte = autcarti.cod_carte
INNER JOIN autori on autori.cod_aut = autcarti.cod_autor
INNER JOIN edituri on edituri.cod_edit = carti.cod_edit
ORDER BY carti.titlu
```

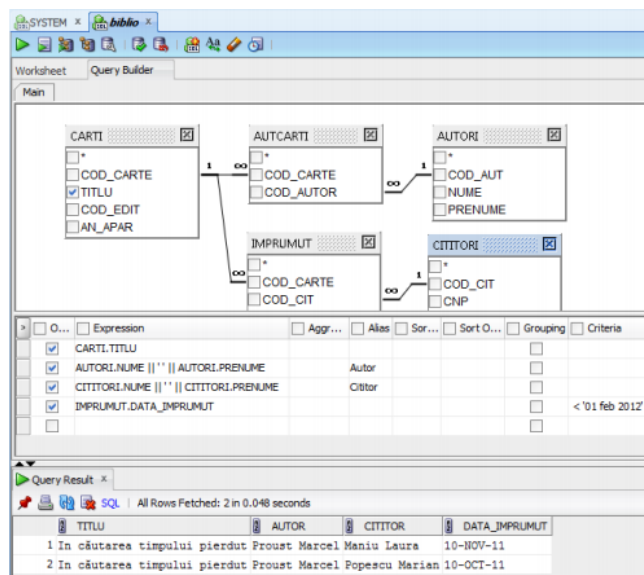
- titlul cărții, numele și prenumele autorului, numele și prenumele cititorului pentru cărțile împrumutate înainte de 01 feb 2011.

Query Result x				
SQL All Rows Fetched: 2 in 0.048 seconds				
TITLU	AUTOR	CITITOR	DATA_IMPRUMUT	
1 In căutarea timpului pierdut	Proust Marcel	Maniu Laura	10-NOV-11	
2 In căutarea timpului pierdut	Proust Marcel	Popescu Marian	10-OCT-11	

```

SELECT carti.titlu, autori.nume || ' ' || autori.prenume as
autor, cititori.nume || ' ' || cititori.prenume as cititor,
imprumut.data_imprumut
FROM CARTI INNER JOIN autcarti
ON carti.cod_carte = autcarti.cod_carte
INNER JOIN autori
ON autori.cod_aut = autcarti.cod_autor
INNER JOIN imprumut
ON carti.cod_carte = imprumut.cod_carte
INNER JOIN CITITORI
ON cititori.cod_cit = imprumut.cod_cit
WHERE imprumut.data_imprumut < '01 feb 2012'

```



1. Afișați salariul maxim, minim, suma salariilor și salariul mediu pentru toți angajații.
Rotunjiți salariul mediu la cel mai apropiat întreg.
2. Afișați salariul maxim, minim, suma salariilor și salariul mediu pentru fiecare funcție.
Rotunjiți Salariul mediu la cel mai apropiat întreg.
3. Determinați numărul de angajați cu aceeași funcție.
4. Determinați numărul de manageri fără a-i afișa.
5. Care este diferența între salariul minim și salariul maxim?
6. Afișați id-ul managerului și salariul celui mai slab plătit angajat al aceluia manager.
Excludeți pe oricine al cărui manager nu este cunoscut și orice grup unde salariul minim este 2000\$ sau mai puțin. Sortați rezultatele în ordinea descrescătoare a salariilor.
7. Afișați pentru fiecare departament numele, locația, numărul de angajați și salariul mediu pe departament. Rotunjiți salariul mediu la 2 zecimale.
8. Afișați numărul total de angajați și apoi numărul de persoane angajate în anii 1995, 1996, 1997 și 1998.
9. Creați o matrice care să afișeze funcția, salariul total pentru acea funcție raportat la departament și salariul total pe acea funcție, pentru departamentele 10, 20 și 30.